Academic Project Documentation

# Enforcing Mandatory Fields Using UI Policies and Migrating Changes With Update Sets in ServiceNow

## Submitted By


| | |
|---|---|
| 6A35198A79FA5B3983C4158BE8D55E64 | HARINI V N |
| 567E0CE444E61F69F3E4051A73E6E132 | KIRIJA C |
| 0282154F5B6DA3995D74445167149FAA | PRIYADHARSHINI D |
| 6B6DE86C89AB920A1C44A7BE9B55FAFC | RAGAVI V |


*Submitted in partial fulfilment of the requirements of the course ServiceNow Platform Administration and IT Service Management*

# 1. Abstract

This project documents the design, implementation, and migration of a UI Policy within the ServiceNow platform, specifically addressing the enforcement of mandatory field completion on the Incident form. The central requirement is that the Priority field must become mandatory whenever an incident's State is changed to "In Progress", thereby preventing agents from advancing an incident without supplying the information necessary for effective prioritisation and SLA management.

The implementation leverages ServiceNow's native UI Policy and UI Policy Action framework, which operates entirely on the client side without custom scripting. All configuration changes are systematically captured in a dedicated Update Set, a version-controlled container that records every platform customisation made during a development session. Once verified, the Update Set is exported as an XML file and imported into a separate ServiceNow instance, where it is previewed for conflicts and committed to apply the identical configuration.

The project demonstrates the complete lifecycle of a low-code ServiceNow configuration: from problem identification and requirement analysis through hands-on implementation, structured migration, and functional validation. The outcome confirms that mandatory field enforcement can be achieved declaratively, that configuration changes can be reliably transported between environments, and that data quality in incident management can be meaningfully improved without writing a single line of server-side code.

The configuration changes are systematically captured within a dedicated Update Set, which serves as a transportable container for all customisations made to the ServiceNow instance. The Update Set is subsequently exported as an XML file and imported into a separate target instance, demonstrating a complete, version-controlled migration workflow. The project validates the effectiveness of both mechanisms through structured testing within the Incident module, confirming correct enforcement behaviour and successful cross-instance configuration transfer.

# 2. Introduction

A fundamental aspect of maintaining data quality within ServiceNow is the enforcement of business rules at the form level. While server-side mechanisms such as Business Rules and Access Controls are commonly employed, they operate on the server after data submission. In contrast, UI Policies are client-side configurations that dynamically modify form behaviour in real time, based on defined conditions. This distinction makes UI Policies particularly effective for enforcing field-level requirements that must be communicated to the user before form submission occurs.

Update Sets are a native ServiceNow mechanism that provides a structured approach to capturing, managing, and transporting configuration changes between instances. In an enterprise ServiceNow deployment, configurations are typically developed in a sub-production development instance, validated in a test or UAT environment, and finally promoted to production. Update Sets underpin this promotion workflow by serving as version-controlled containers that record every customisation made during a development session.

This project integrates both capabilities to address a common data governance challenge in incident management: the absence of mandatory priority classification when incidents are actively being worked on. By implementing a UI Policy to enforce the Priority field and capturing the change within an Update Set for subsequent migration, the project demonstrates a complete, professionally structured approach to ServiceNow platform administration.

## 2.1 The ServiceNow Platform

ServiceNow is a cloud-based IT Service Management (ITSM) platform designed to help organisations automate, standardise, and optimise their IT workflows. Built on a multi-tenant architecture, it provides a unified system of record for managing incidents, service requests, changes, problems, assets, and configurations. Organisations across industries deploy ServiceNow to reduce operational friction, enforce governance policies, and improve the consistency of IT service delivery. The platform's no-code and low-code configuration capabilities make it accessible to administrators without deep programming backgrounds, while its scripting engine accommodates complex enterprise customisations.

## 2.2 Incident Management in ServiceNow

The Incident Management module is one of the most widely used applications within ServiceNow. It provides a structured workflow for logging, categorising, assigning, and resolving IT issues reported by end users or detected by monitoring systems. Incidents move through a defined lifecycle — from New to In Progress, On Hold, Resolved, and Closed — with each state transition representing a meaningful step in the resolution process. Accurate data entry at each transition point is critical for SLA compliance, reporting accuracy, workload distribution, and effective escalation management.

## 2.3 Business Rule Enforcement and UI Policies

ServiceNow provides multiple mechanisms for enforcing business rules. Server-side mechanisms such as Business Rules and Data Policies evaluate conditions and constraints after form submission, which can result in confusing error messages post-save. UI Policies, by contrast, operate entirely on the client side, responding dynamically to field changes as the user interacts with the form. They can make fields mandatory, read-only, or visible/hidden based on configurable conditions, providing immediate, real-time guidance to the user before submission occurs. This makes UI Policies the most appropriate tool for enforcement requirements that benefit from immediate user feedback.

## 2.4 Update Sets and Configuration Migration

In a standard ServiceNow deployment, the platform is instantiated across multiple environments: a development instance where configurations are built and tested, a UAT or staging instance for validation, and a production instance for live operations. Update Sets are the native mechanism for transporting configuration changes between these environments. When an Update Set is designated as active ("current"), ServiceNow records every subsequent configuration change — including UI Policies, Business Rules, and form layouts — into that container. The completed Update Set can then be exported as XML, transferred, and imported into any target instance. This approach ensures consistency, provides an audit trail, and eliminates the risk of manual re-configuration errors in target environments.

## 3. Problem Statement

In organisations relying on ServiceNow for IT incident management, a recurring data quality issue arises when support agents transition an incident to the "In Progress" state without completing all fields that are contextually critical at that stage. The Priority field is one such field: it defines the urgency and business impact of the incident, drives SLA timer activation, determines queue routing in automated workflows, and informs management dashboards and escalation logic. When Priority is absent at the point of active case management, the downstream consequences include incorrect SLA calculations, inaccurate reporting metrics, and potential breaches of service commitments.

Compounding this issue is the challenge of migrating any corrective configuration to production. Without a disciplined Update Set workflow, administrators risk applying configurations manually across environments, leading to discrepancies between development and production instances, inadequate change records, and difficulty rolling back if an issue arises. This project addresses both the enforcement gap and the migration challenge through a structured, documented implementation.

## 4. Objectives

The following objectives define the scope and success criteria of this project:

- Design and implement a UI Policy on the ServiceNow Incident table that enforces the Priority field as mandatory when the incident State field is set to "In Progress".
- Configure a corresponding UI Policy Action that applies the mandatory attribute specifically to the Priority field under the defined condition.
- Create a dedicated Update Set prior to implementation to ensure that all configuration changes are systematically captured within a single, named container.
- Verify that the Update Set correctly records the UI Policy and UI Policy Action as customer update entries.
- Export the completed Update Set from the source instance as an XML file for transport.
- Import the Update Set into a separate target ServiceNow instance using the Retrieved Update Sets module.

## 5. Scope of the Project

The scope of this project is confined to the ServiceNow Incident Management module, specifically the enforcement of a single mandatory field condition using a UI Policy and the migration of that configuration using an Update Set. The implementation targets any ServiceNow instance running on the Rome release or later, including Personal Developer Instances (PDIs) provisioned through the ServiceNow Developer Program.

The solution applies to all users who interact with the Incident form and have the ability to modify the State field. It does not require changes to any other module, table, or application, and it does not rely on custom scripts, business rules, or workflow modifications. The Update Set migration scope covers the source and one target instance, demonstrating the standard single-hop promotion pattern used in most enterprise deployments.

## 6. Tools and Technologies Used

| Tool / Technology | Purpose |
|---|---|
| ServiceNow Platform (PDI) | The cloud-based ITSM platform on which all development, configuration, and testing activities are performed. A Personal Developer Instance is used to provide a safe, isolated development environment. |
| Incident Table (incident) | The application table on which the UI Policy is created. It represents IT support cases and contains the State and Priority fields central to this project. |
| UI Policies | Client-side configuration objects that dynamically adjust form field behaviour — such as mandatory status, visibility, and read-only state — based on conditions defined by the administrator. |
| UI Policy Actions | Child records of a UI Policy that specify the exact field-level constraint to enforce. Each action targets one field and defines its mandatory, visible, or read-only setting. |
| Update Sets | Version-controlled configuration containers that record all customisations made while the set is designated as current. Used to transport configurations between ServiceNow instances. |
| XML Export / Import | The serialisation format used to export an Update Set from the source instance and import it into the target instance for preview and commit. |
| Web Browser | Modern web browser (Google Chrome recommended) used to access the |

# 7. Project Phases

## Phase 1 — Identification Phase

The Identification Phase establishes the problem context and justifies the need for a technical solution. The specific issue — incidents progressing to the "In Progress" state without a Priority value — is identified through analysis of common incident data quality failures in ServiceNow environments. The suitability of a UI Policy as a client-side enforcement mechanism is confirmed, and the need for a structured Update Set migration workflow is articulated. The phase concludes with a formally defined project scope and a clear statement of the business value the solution will deliver.

## Phase 2 — Requirement Analysis

Requirement Analysis translates the identified problem into specific, testable requirements. The primary functional requirement is defined: when an incident's State field is set to "In Progress", the Priority field must be marked as mandatory and must prevent form submission if empty. Technical requirements include access to a ServiceNow instance with administrative privileges, the ability to create UI Policies on the Incident table, and access to the Update Sets module. The requirement for the solution to operate without custom scripting is also documented as a non-functional requirement, ensuring long-term maintainability and upgrade compatibility.

## Phase 3 — Project Planning Phase

The Planning Phase organises the implementation into a seven-step sequence with defined inputs, outputs, and acceptance criteria for each step. Key milestones are identified: Update Set creation and activation, successful UI Policy configuration, Update Set capture verification, XML export, target instance import, and post-commit validation. Risks are assessed — including PDI hibernation, Dot Walk resolution failures, and import conflicts — and mitigation strategies are documented. The planning phase ensures that the implementation proceeds systematically and that each step can be independently verified before proceeding to the next.

## Phase 4 — Project Design Phase

The Design Phase defines the logical structure of the solution. The UI Policy is designed with a single, precise condition: State equals "In Progress" (integer value 2 in the ServiceNow incident.state field). The UI Policy Action is designed to target the priority field exclusively, with the Mandatory attribute set to True, and both Visible and Read Only set to "Leave alone" to avoid unintended side effects. The Update Set is designed as a project-scoped container with a descriptive naming convention. The expected migration workflow is documented as a linear sequence: capture, complete, export, import, preview, commit, and validate.

## Phase 5 — Project Development Phase

The Development Phase encompasses the hands-on implementation of all configured components within the ServiceNow instance. All seven implementation steps are executed in sequence, with screenshots captured at each key stage as evidence of correct configuration. The phase includes the creation of the Update Set, the configuration of the UI Policy and its associated Action, and the full export-import migration cycle. The phase concludes when the UI Policy is confirmed as active in the target instance and the Transform Log — in this context, the Update Set commit log — reports a successful outcome.

## Phase 6 — Project Documentation Phase

The Documentation Phase produces this complete academic report, consolidating all phases, implementation steps, testing procedures, results, and supporting evidence into a single structured document. The report is written in formal academic language, uses a consistent heading hierarchy, maintains justified paragraph alignment, and includes screenshot placeholders at all relevant implementation points. The documentation is structured for submission in an academic or internship context and is sufficiently detailed to serve as a reproducible reference guide for any administrator wishing to replicate the configuration.

## Phase 7 — Project Demonstration Phase

The Demonstration Phase presents the working solution to an examiner, supervisor, or review panel. The demonstration covers four scenarios: opening an incident and changing the State to "In Progress" to trigger the mandatory Priority indicator; attempting to save without a Priority value to confirm that the validation prevents submission; completing the Priority field and saving successfully; and presenting the Update Set XML file alongside the post-commit UI Policy list in the target instance. All four scenarios are executed live or captured in a video walkthrough, confirming that the solution meets every stated objective.

# 8. System Architecture and Workflow

## 8.1 UI Policy Execution Workflow

When a user opens an Incident record in ServiceNow, the platform loads all UI Policies associated with the Incident table into the browser. Each policy's condition is evaluated client-side as the user modifies form fields. In this project, when the user changes the State field to "In Progress", the UI Policy condition becomes true and the browser immediately applies the UI Policy Action: the Priority field is marked as mandatory, displaying a red asterisk and triggering a validation error if the user attempts to save without completing the field. If the State is subsequently changed to any other value, the condition becomes false and the mandatory indicator is removed, demonstrating the dynamic, reversible nature of UI Policy enforcement.

## 8.2 Update Set Capture and Migration Workflow

The Update Set workflow follows a linear, multi-stage process. First, an administrator creates a new Update Set and designates it as current. All subsequent configuration changes — including the creation of the UI Policy record and its UI Policy Action — are automatically recorded as customer update entries within the active Update Set. Once all changes are complete, the Update Set's state is changed to Complete, which prevents further capture and prepares it for export. The administrator uses the Export to XML function to download the Update Set as a self-contained XML file.

**MIGRATION WORKFLOW OVERVIEW**

Create Update Set → Make Current → Configure UI Policy → Add UI Policy Action

↓

Verify Capture → Set Status to Complete → Export to XML

↓

Import XML to Target Instance → Preview → Commit → Validate

# 9. Implementation Steps

**STEP 1**

### Create a Dedicated Update Set

Navigate to System Update Sets → Local Update Sets and click New. In the Name field, enter a descriptive identifier such as "UI Policy – Enforce Priority on In Progress Incidents". Set the Description field to document the purpose of this Update Set for future reference. Leave the State as "In Progress" and click Submit to save the record.

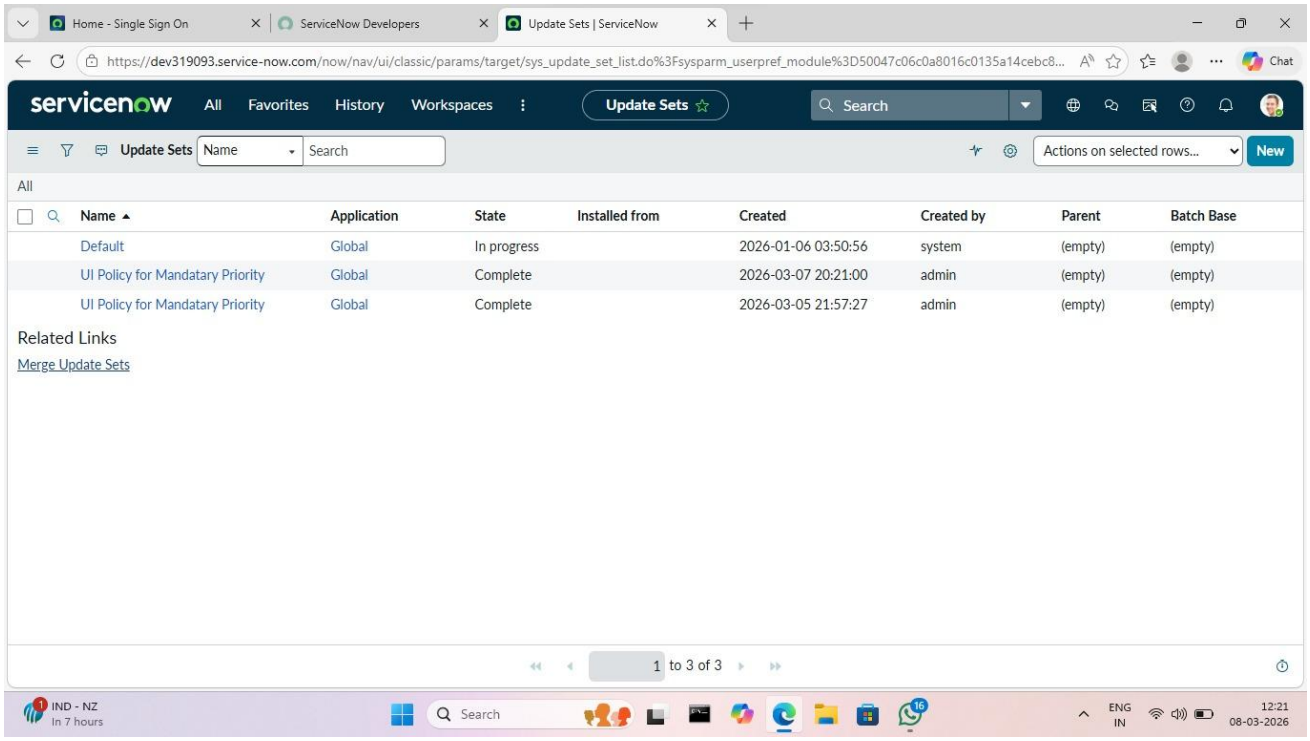


*Figure 1 – New Update Set record showing Name, Description, and State fields*

STEP
2

## Designate the Update Set as Current

Open the Update Set record that was just created. In the form header, click the Make Current button. Observe that the ServiceNow banner at the top of the interface now displays the Update Set name as the active set. This confirms that all subsequent configuration changes will be captured within this Update Set. Verify the active set name before proceeding.

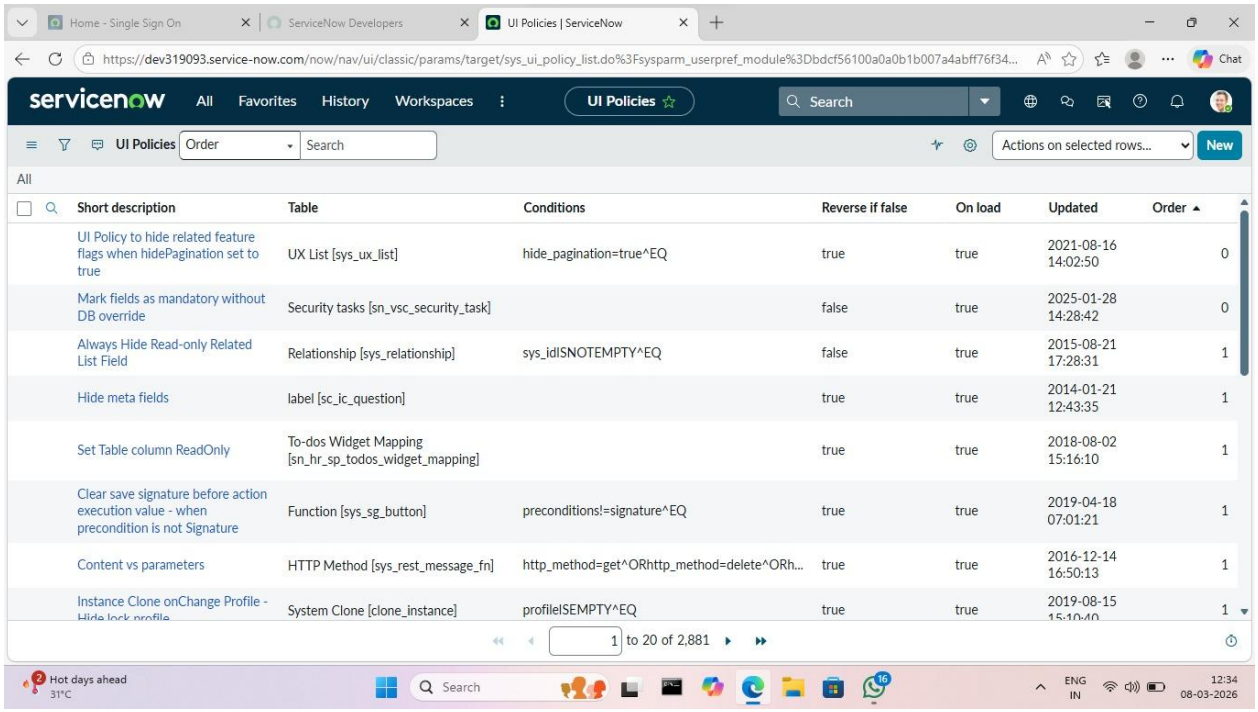


*Figure 2 – ServiceNow instance header confirming the active Update Set name*

| STEP 3 | **Create a New UI Policy**<br><br>Navigate to System UI → UI Policies and click New. In the Table field, select Incident [incident] from the list. Enter a clear Name such as "Enforce Priority – In Progress State". Ensure that the Active checkbox is checked. Also check the On load checkbox to ensure the policy evaluates when the Incident form is first opened by a user, not only when a field is changed during the session. |
|---|---|

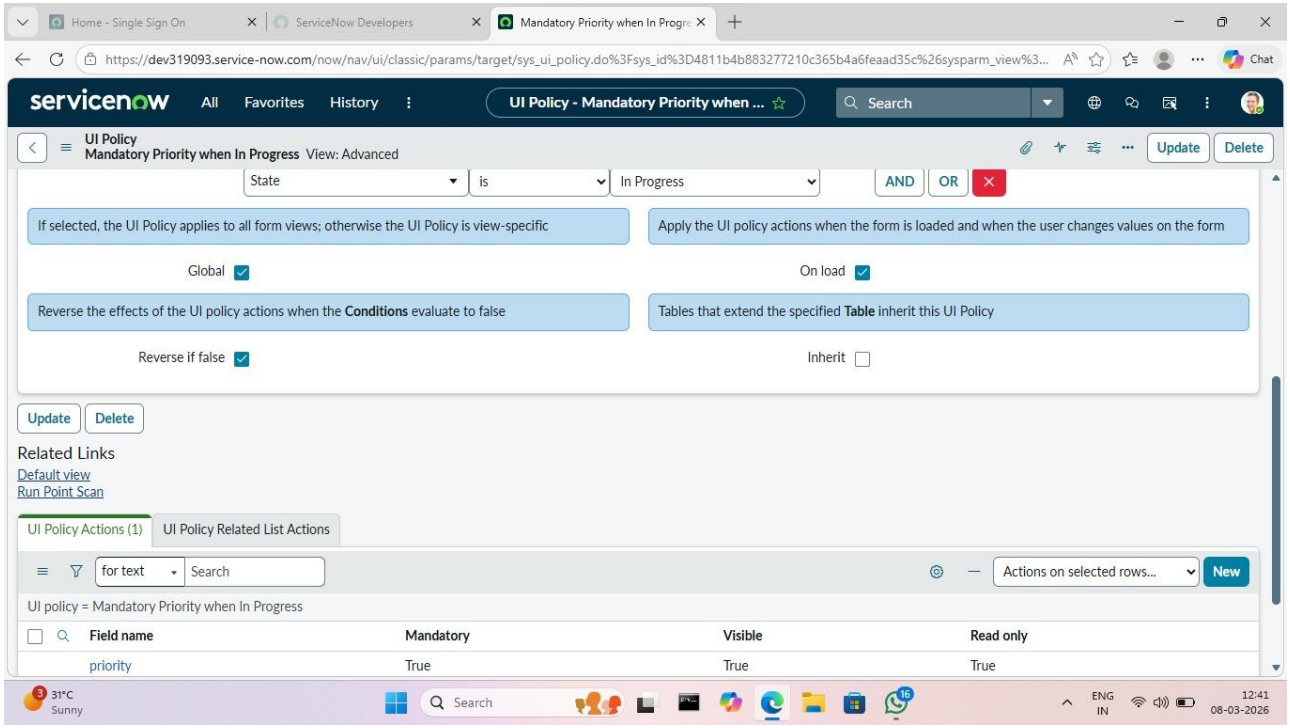


*Figure 3 – UI Policy form showing Table set to Incident, Name, Active, and On load fields*

STEP
4

## Configure the UI Policy Condition

In the Conditions section of the UI Policy form, click the filter builder to add a condition. Set the Field to State, the Operator to is, and the Value to In Progress. The condition string should read: State is In Progress. This ensures the policy activates exclusively when the State field value equals In Progress and remains inactive for all other state values. Click Submit to save the condition.

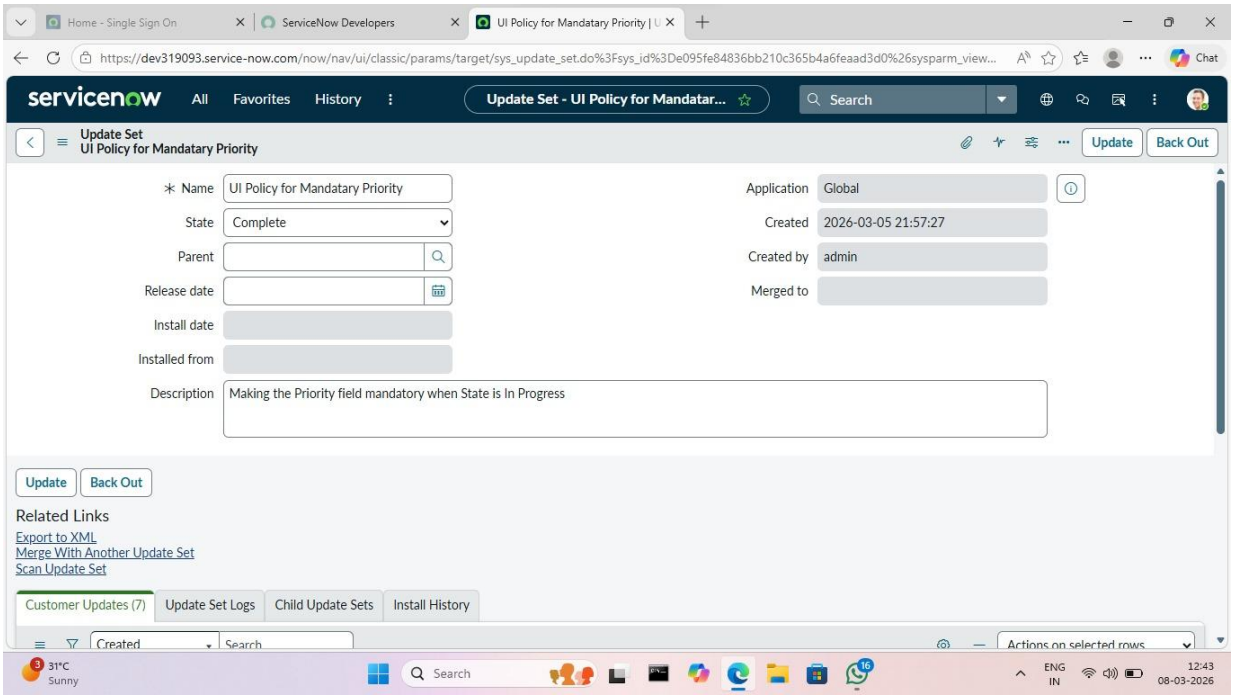


*Figure 4 – UI Policy Conditions section showing: State | is | In Progress*

STEP
5

## Add a UI Policy Action for the Priority Field

Scroll to the UI Policy Actions related list at the bottom of the UI Policy form and click New. In the Field field, select Priority. Set the Mandatory dropdown to True. Set both the Visible and Read Only dropdowns to Leave alone to ensure only the mandatory attribute is applied without affecting field visibility or editability. Click Submit to save the UI Policy Action record.

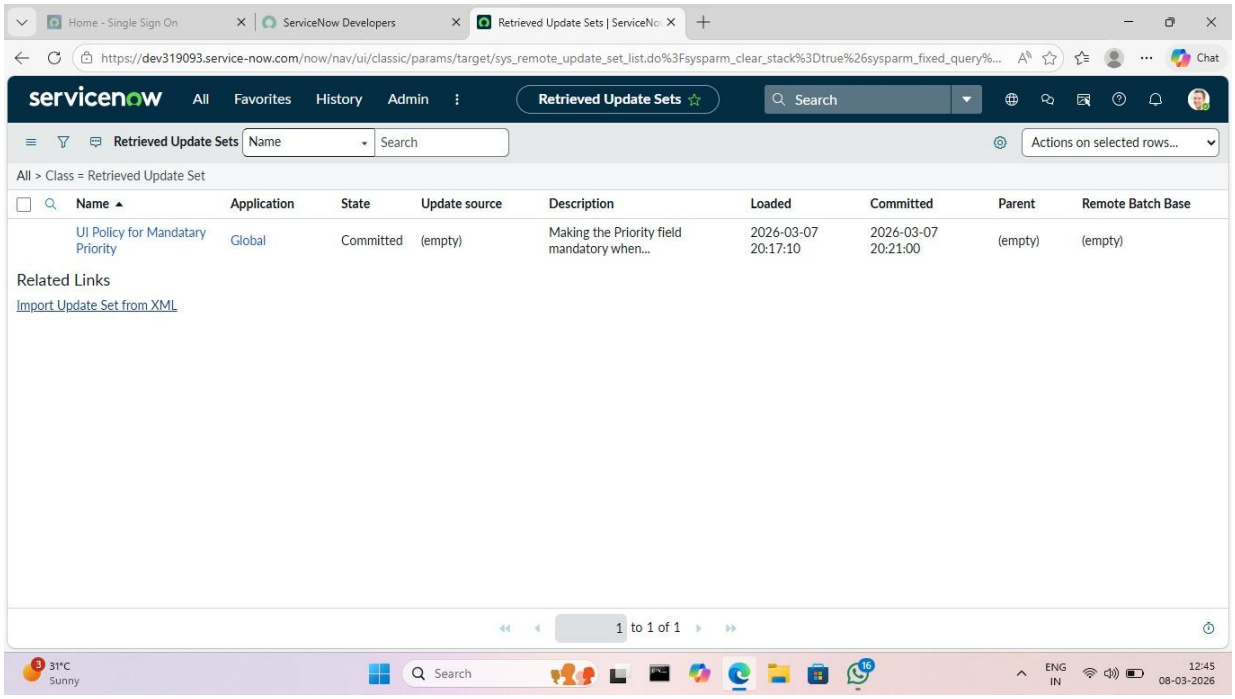


*Figure 5 – UI Policy Action form: Field = Priority, Mandatory = True, Visible = Leave alone, Read Only = Leave alone*

STEP
6

## Save and Activate the UI Policy

Return to the parent UI Policy record and confirm that the UI Policy Action is visible in the related list. Verify that the Active checkbox on the UI Policy record is checked. If the form is in edit mode, click Update to save all changes. The UI Policy is now active and will enforce the Priority mandatory rule on the Incident form whenever the State is In Progress.

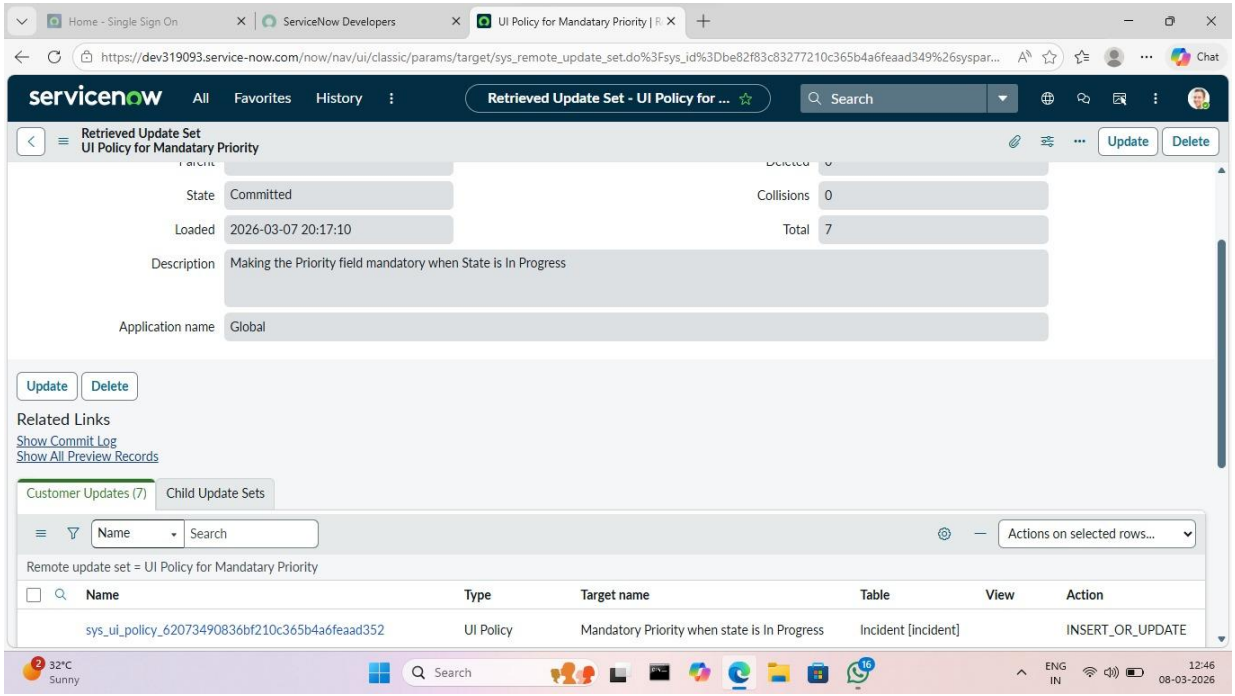


*Figure 6 – UI Policy record in active state with UI Policy Action visible in related list*

STEP 7

## Verify Update Set Capture

Navigate to System Update Sets → Local Update Sets and open the active Update Set. Scroll to the Customer Updates related list at the bottom of the record. Confirm that two entries are present: one corresponding to the UI Policy record and one corresponding to the UI Policy Action record. Both entries confirm that the configuration changes have been successfully captured and are ready for export.

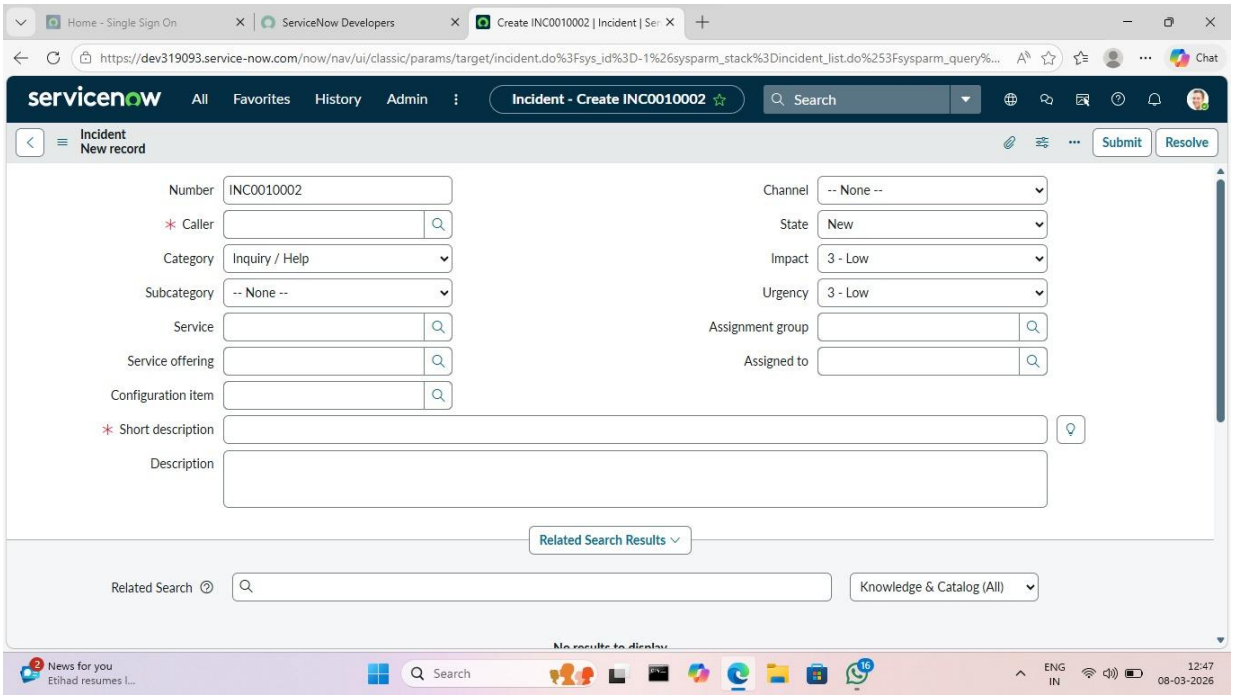


*Figure 7 – Customer Updates related list showing UI Policy and UI Policy Action entries*

## 10. Advantages

| Advantage | Explanation |
|---|---|
| Improved Data Accuracy | Mandatory field enforcement at the point of state transition eliminates a common source of incomplete incident records, improving the quality of operational data and the reliability of reports and dashboards. |
| Real-Time User Feedback | Because UI Policies execute client-side, feedback is immediate. The mandatory indicator appears the moment the State is changed, before the user attempts to save, reducing confusion and rework. |
| No Custom Scripting Required | The entire enforcement mechanism is achieved through declarative, no-code configuration. This reduces development complexity, lowers the risk of scripting errors, and ensures compatibility with future ServiceNow upgrades. |
| Better Workflow Enforcement | By tying field requirements to specific lifecycle transitions, the platform actively guides agents through the correct workflow, reducing the need for supervisory review of incomplete records. |
| Easy Configuration Migration | The Update Set framework provides a structured, repeatable migration pathway. The same XML file can be imported into multiple target instances, ensuring consistent enforcement across development, test, and production environments. |
| Auditability | Every configuration change captured in an Update Set is traceable. The customer update log records who made the change, when, and what was modified, supporting governance and change management requirements. |
| Low-Code Implementation | The solution requires no programming background to implement or maintain. Any ServiceNow administrator with basic platform knowledge can replicate, modify, or extend the configuration using the standard administrative interface. |

## 11. Limitations

- UI Policies are client-side only and execute within the browser. They can be bypassed by API calls, import operations, or any process that creates or updates records without loading the ServiceNow form. For server-side enforcement that cannot be bypassed, a Data Policy or Business Rule must be used in addition to the UI Policy.
- UI Policies apply only to the form view in the standard ServiceNow interface. They do not affect list editing, mobile interfaces (without additional configuration), or integrations that write data directly to the database layer.
- Update Sets capture configuration changes but do not capture data. Changes to records in operational tables — such as incident data, user records, or knowledge articles — are not included in an Update Set and must be migrated by other means if required.
- Personal Developer Instances used for development may enter a hibernation state after a period of inactivity. If the instance hibernates during an active development session, the Update Set may need to be reviewed to confirm that all changes were captured before the instance went offline.

## 12. Future Enhancements

- Implement a complementary Data Policy on the Incident table to enforce the Priority mandatory rule at the server side, ensuring that the constraint is applied even for API-based or import-based record updates that bypass the form entirely.
- Extend the UI Policy framework to enforce additional contextually relevant fields at other state transitions. For example, making the Resolution Notes and Resolution Code fields mandatory when State changes to Resolved, and requiring Category and Subcategory before an incident can be Assigned.
- Configure an automated notification or task assignment trigger using a Flow Designer flow that fires when an incident transitions to "In Progress" without a Priority value, as a compensating control for the API bypass scenario described in Section 16.
- Explore the use of Automated Test Framework (ATF) test suites to automate the regression testing of UI Policies across multiple Incident scenarios, reducing the time required for manual validation after each Update Set migration.

## 13. Conclusion

This project has successfully demonstrated two core ServiceNow administration competencies: the dynamic enforcement of mandatory fields using UI Policies, and the structured migration of configuration changes using Update Sets. Both capabilities were applied to a realistic and practically significant use case — ensuring that every incident reaching the "In Progress" state carries a Priority value, which is essential for SLA compliance, workload management, and accurate reporting.

The UI Policy implementation confirmed that meaningful business rule enforcement can be achieved through entirely declarative, no-code configuration. The policy operates in real time, responds dynamically to user interactions, and deactivates automatically when its condition is no longer satisfied. This approach is maintainable, upgrade-safe, and accessible to administrators without scripting expertise.

The Update Set workflow demonstrated that ServiceNow's native configuration transport mechanism is reliable, auditable, and straightforward to operate. The complete export-import-preview-commit cycle produced identical behaviour in the target instance without manual re-configuration, validating the Update Set as the correct tool for enterprise-scale configuration management.

Collectively, the project illustrates a professional, methodical approach to ServiceNow platform administration: identifying a data quality problem, designing a precise, minimal solution, implementing it within a controlled development environment, and migrating it to a target instance through a documented, reversible process. This methodology is directly applicable to real-world enterprise ServiceNow deployments and represents a foundational competency for any practitioner working in the ServiceNow ecosystem.

*— End of Project Documentation —*